

Packet Scheduling Algorithms
(Algoritmos de Escalonamento de Pacotes)

Modelação e Desempenho de Redes e Serviços

Prof. Amaro de Sousa (asou@ua.pt)

DETI-UA, 2026/2027

Congestion in point-to-point links

- Consider a packet switched network composed by multiple point-to-point links.
- On each outgoing interface of each link, there is (at least) one packet queue to temporarily store incoming packets when they arrive and the link is transmitting another packet.
- During time, each link flips between:
  - *non-congested time periods*, time periods without queued packets, i.e., all packets arrive at a rate such that they start being transmitted when they arrive,
  - *congested time periods*, time periods with queued packets, i.e., packets arrive at a rate such that they must be stored in a queue (because, for example, the link is transmitting another packet).

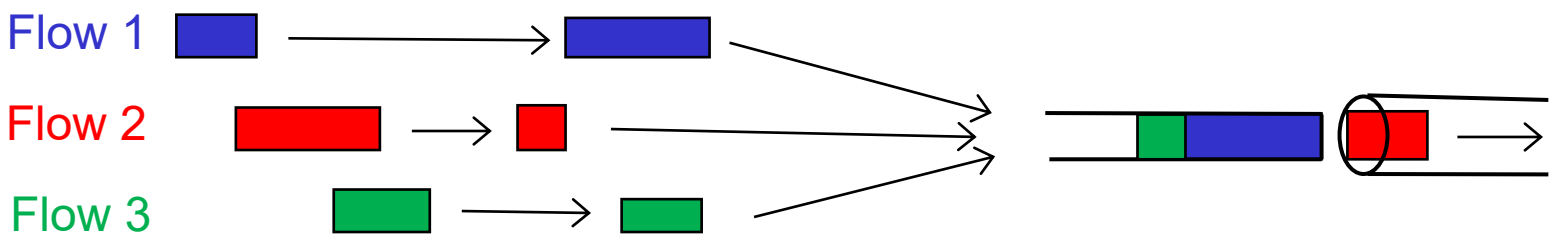
Congestion in point-to-point links

- The average packet delay suffered by the packets on a point-to-point link is the sum of two components (ignoring the propagation delay):
  - the average packet transmission time, which is only characterized by the packet size statistics and the capacity of the link,
  - the average packet queuing delay.
- Therefore:
  - in *non-congested time periods*, the average packet delay is only given by the average packet transmission time,
  - in *congested time periods*, the average packet delay is the sum of the two (above described) delay components.
- The avg. packet queuing delay (which happens in congested time periods) is influenced by the Packet Scheduling Algorithm configured in the outgoing interface of the link

Packet scheduling algorithms

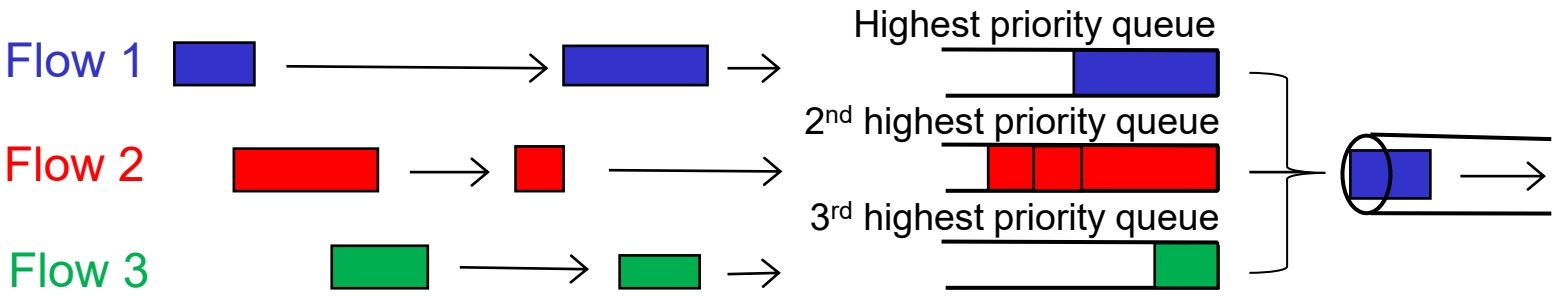
- On the outgoing interface of each link, the Packet Scheduling Algorithm:
  - decides the order by which the queued packets of different flows are transmitted and when each packet starts being transmitted,
  - therefore, it can provide different average packet queueing delays to different packet flows.
- The packet scheduling algorithms can be classified as:
  - *work conserving algorithms*: the link is free (i.e., not transmitting any packet) only if no packet is in the queue(s),
  - *non work conserving algorithms*: the link can be free with packets waiting in the queue(s).
- In the previous modules:
  - FIFO is a work conserving algorithm,
  - the algorithm for flows with assigned resources is a non work conserving algorithm.

First-in-First-Out (FIFO)



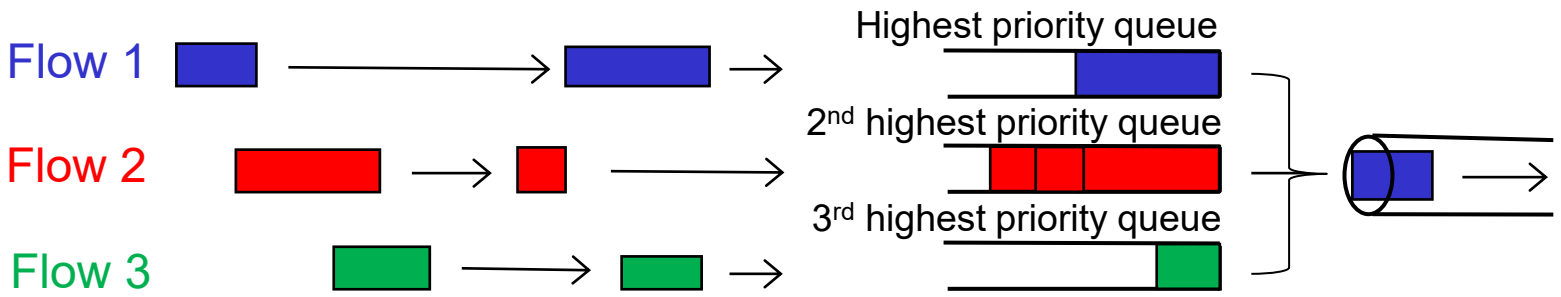
- Packets of all flows are temporarily stored in a single queue and transmitted by their arriving order.
- FIFO is the simplest algorithm as it does not require packet classification into different classes.
- It does not allow delay differentiation between flows (already seen that the avg. packet queuing delay is equal for all flows).
- In congested time periods:
  - the avg. packet queuing delay of each flow is dependent on the traffic of the other flows.
  - flows with *n* times more arriving throughput receive *n* times more transmission rate.

Priority Queuing (PQ)



- There is a queue per priority (PQ requires packet classification).
- Packets of a given priority are always transmitted before packets of all lower priorities (packets of the same priority are transmitted by their arriving order).
- It allows delay differentiation between flows of different priorities (the avg. packet queuing delay is lower for higher priority flows).
- In congested time periods, the avg. packet queuing delay of each priority is dependent on the traffic of all lower priorities:
  - in particular, flows of higher priority can completely prevent the lower priority flows to be transmitted (the starvation problem).

Priority Queuing (PQ)



- When all queues are very large, the avg. packet queuing delay of each priority flow is given by a *M*/*G*/1 model with priorities.
- Consider the packet flow of priority *k*, $1 \le k \le n$, defined by:
  - packet arriving rate: λ_k
  - average and 2nd moment of the packet transmission time: $E[S_k]$ and $E[S_k^2]$
- The avg. packet queuing delay of flow with priority *k* is:

$$W_{Qk} = \begin{cases} \dfrac{\sum_{i=1}^{n}(\lambda_i E[S_i^2])}{2(1-\rho_1)} & , k = 1 \\ \dfrac{\sum_{i=1}^{n}(\lambda_i E[S_i^2])}{2(1-\rho_1-\cdots-\rho_{k-1})(1-\rho_1-\cdots-\rho_k)} & , k > 1 \end{cases} \qquad \rho_k = \lambda_k E[S_k]$$

Validity condition: $\rho_1 + \cdots + \rho_n < 1$

Example 1

Consider a link of 10 Mbps with a very large queue supporting 2 packet flows: flow A of 1 Mbps and flow B of 6 Mbps. The packets of both flows are exponentially distributed with average 1000 bytes. Determine the average packet delay of each flow when:

(a) the flows are served with a single FIFO queue,

(b) the flows are served with PQ and flow A has higher priority than flow B.

Example 1 – solution of (a)

Consider a link of 10 Mbps with a very large queue supporting 2 packet flows: flow A of 1 Mbps and flow B of 6 Mbps. The packets of both flows are exponentially distributed with average 1000 bytes. Determine the average packet delay of each flow when:

(a) the flows are served with a single FIFO queue,

Since the packet sizes of both flows are exponentially distributed with the same average size, this system is modelled by a *M*/*M*/1 system.

$$\mu = \frac{10\times10^6}{8\times1000} = 1250 \text{ pps} \qquad \lambda_A = \frac{1\times10^6}{8\times1000} = 125 \text{ pps} \qquad \lambda_B = \frac{6\times10^6}{8\times1000} = 750 \text{ pps}$$

$$W_A = W_B = \frac{1}{\mu - \lambda} = \frac{1}{\mu - (\lambda_A + \lambda_B)} = \frac{1}{1250 - (125 + 750)} = 2.67\times10^{-3} = 2.67 \text{ ms}$$

Example 1 – solution of (b)

Consider a link of 10 Mbps with a very large queue supporting 2 packet flows: flow A of 1 Mbps and flow B of 6 Mbps. The packets of both flows are exponentially distributed with average 1000 bytes. Determine the average packet delay of each flow when:

(b) the flows are served with PQ and flow A has higher priority than flow B.

$$\lambda_A = \frac{1\times10^6}{8\times1000} = 125 \text{ pps}$$

$$\lambda_B = \frac{6\times10^6}{8\times1000} = 750 \text{ pps}$$

$$\mu_A = \mu_B = \mu = \frac{10\times10^6}{8\times1000} = 1250 \text{ pps}$$

$$E[S_A] = E[S_B] = \frac{1}{\mu} = \frac{1}{1250} \text{ seg.}$$

$$E[S_A^2] = E[S_B^2] = \frac{2}{\mu^2} = \frac{2}{1250^2} \text{ seg.}^2$$

Recall that: $E[S^2] = Var[S] + (E[S])^2$

$$W_{Qk} = \begin{cases} \dfrac{\sum_{i=1}^{n}(\lambda_i E[S_i^2])}{2(1-\rho_1)} & ,k=1 \\ \dfrac{\sum_{i=1}^{n}(\lambda_i E[S_i^2])}{2(1-\rho_1-\cdots-\rho_{k-1})(1-\rho_1-\cdots-\rho_k)} & ,k>1 \end{cases} \qquad \rho_k = \lambda_k E[S_k]$$

$$W_A = \frac{\lambda_A \times E[S_A^2] + \lambda_B \times E[S_B^2]}{2\times(1-\rho_A)} + E[S_A] = 1.42 \text{ ms}$$

$$W_B = \frac{\lambda_A \times E[S_A^2] + \lambda_B \times E[S_B^2]}{2\times(1-\rho_A)\times(1-\rho_A-\rho_B)} + E[S_B] = 2.87 \text{ ms}$$

Notion of fairness in packet scheduling algorithms

When a link is in a congested time period, a fair packet scheduling algorithm should assign the scarce resources of the link to different flows with equal rights but with different resource requirements.

Ideally, the assignment of the resources should be done using the **max-min fairness principle**, which states that:

- the resources are assigned to the flows ordered in an increasing way by their requirements;
- none of the flows is assigned with more resources than the ones they require;
- the flows whose requirements cannot be fully satisfied are assigned with the same amount of resources.

Max-min fairness with equal rights

Consider:

- a set of flows 1, 2, ..., n with resource requirements x_1, x_2, ..., x_n ordered by their requirements ($x_1 \leq x_2 \ldots \leq x_n$);
- a link with capacity C.

The capacity is assigned to each flow in the following way:

- Initially all flows can be assigned with $d = C/n$
- d is lower than x_1?
  - If yes, all flows 1, 2, …, n are assigned with d
  - If not, flow 1 is assigned with x_1 and all other flows 2, 3, …, n can be assigned with $d = d + (d - x_1)/(n - 1)$
- d is lower than x_2?
  - If yes, all flows 2, 3, …, n are assigned with d
  - If not, flow 2 is assigned with x_2 and all other flows 3, 4, …, n can be assigned with $d = d + (d - x_2)/(n - 2)$
- and so on…

Example 2

Consider a link with a capacity of 128 Mbps and 4 traffic flows whose arriving throughput is 8, 36, 48 and 128 Mbps. Determine the link throughput that should be assigned to each flow when all flows have equal rights.

i) Flow 1 can be assigned with 128/4 = 32 Mbps.

Since flow 1 wants less than 32 Mbps, flow 1 is assigned with 8 Mbps. There are 32 – 8 = 24 Mbps left.

ii) Flow 2 can be assigned with 128/4 + 24/3 = 40 Mbps.

Since flow 2 wants less than 40 Mbps, flow 2 is assigned with 36 Mbps. There are 40 – 36 = 4 Mbps left.

ii) Flow 3 can be assigned with 128/4 + 24/3 + 4/2 = 42 Mbps.

Since flow 3 wants more than 42 Mbps, flow 3 and flow 4 are assigned with 42 Mbps each.

Max-min fairness with different rights

In the previous case, all flows had the same rights.

When the different flows have different rights, each flow is assigned (by the network operator) with a weight value proportional to its right.

Then, the resources are assigned according to the **weighted max-min fairness principle**. In this case:

- The resources are assigned to the flows ordered in an increasing way by their requirements, normalized by their weight values.
- None of the flows is assigned with more resources than the ones they require.
- The flows whose requirements are not fully satisfied are assigned a resource quantity proportional to their weight values.

Example 3

Consider a link with a capacity of 128 Mbps and 4 traffic flows whose arriving throughput is 8, 36, 48 and 128 Mbps. Determine the link throughput that should be assigned to each flow when the flows have the weight values 1, 1, 3 and 3, respectively.

i) Flow 1 : $1/(1+1+3+3) \times 128 = 16$ Mbps

Flow 2 : $1/(1+1+3+3) \times 128 = 16$ Mbps

Flow 3 : $3/(1+1+3+3) \times 128 = 48$ Mbps

Flow 4 : $3/(1+1+3+3) \times 128 = 48$ Mbps

8 Mbps are assigned to flow 1 (<16 Mbps) and 48 Mbps to flow 3.

There are (16 – 8) + (48 – 48) = 8 Mbps left.

ii) Flow 2 : $16 + 1/(1+3) \times 8 = 18$ Mbps

Flow 4 : $48 + 3/(1+3) \times 8 = 54$ Mbps

18 Mbps are assigned to flow 2 (<36 Mbps) and 54 Mbps are assigned to flow 4 (<128 Mbps).

Comparison of the results of Examples 2 and 3

Link capacity: 128 Mbps

Flows:	1	2	3	4
Throughput (Mbps):	8	36	48	128
Weights:	1	1	1	1
Assigned throughput (Mbps):	**8**	**36**	42	42
Weights:	1	1	3	3
Assigned throughput (Mbps):	**8**	18	**48**	54

- When all flows have equal weights, flows 1 and 2 get their total throughput because they are the one with less throughput
- When the weight values of flows 3-4 are three times the weight values of flows 1-2, flows 3-4 get higher throughputs at the cost of flow 2 which gets less throughput than its need

Packet scheduling algorithms providing weighted max-min fairness

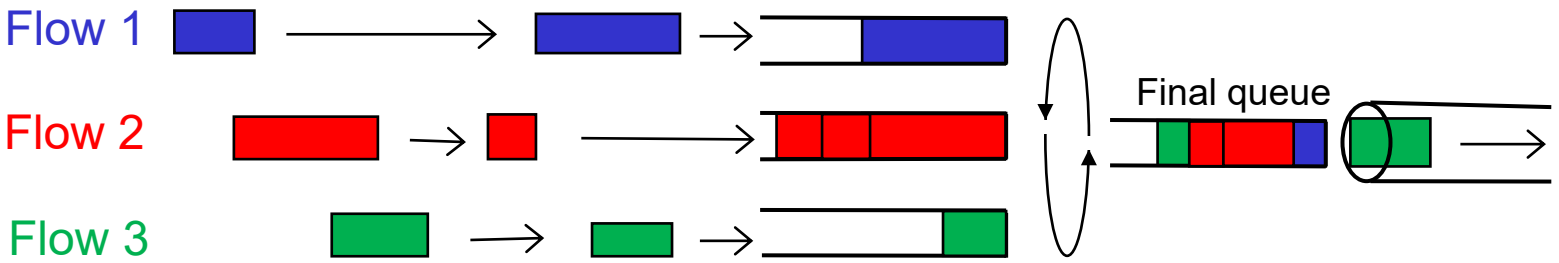
- Weighted max-min fairness provides a minimum throughput to each packet flow in congested time periods.
- In a link of capacity *C* and supporting flows i = 1, 2, …, N with weight values w_1, w_2, …, w_N, the minimum guaranteed transmission rate r_i for flow n is:

$$r_i = \frac{w_i}{w_1 + w_2 + \cdots + w_N} \times C$$

(this value is guaranteed when all flows are active and is higher when not all flows are active)

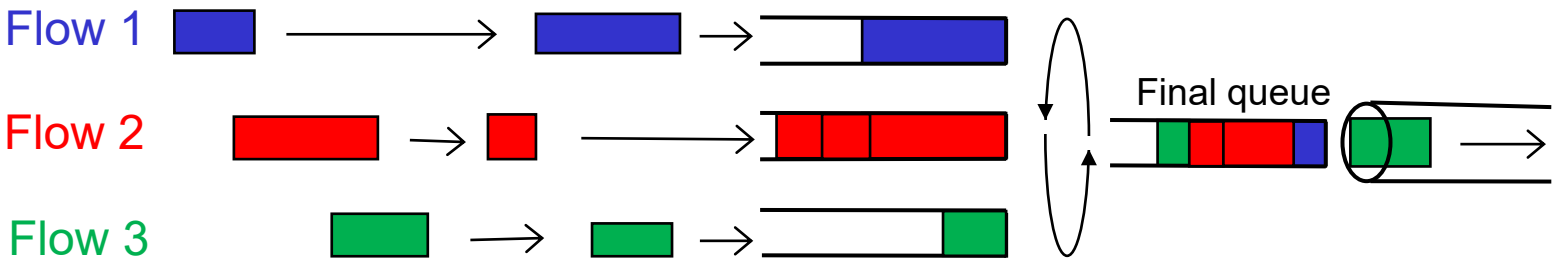
- So, unlike FIFO and PQ, the packet arriving rate of each flow only influences its avg. packet queuing delay in congested time periods.

Algorithms of type round robin



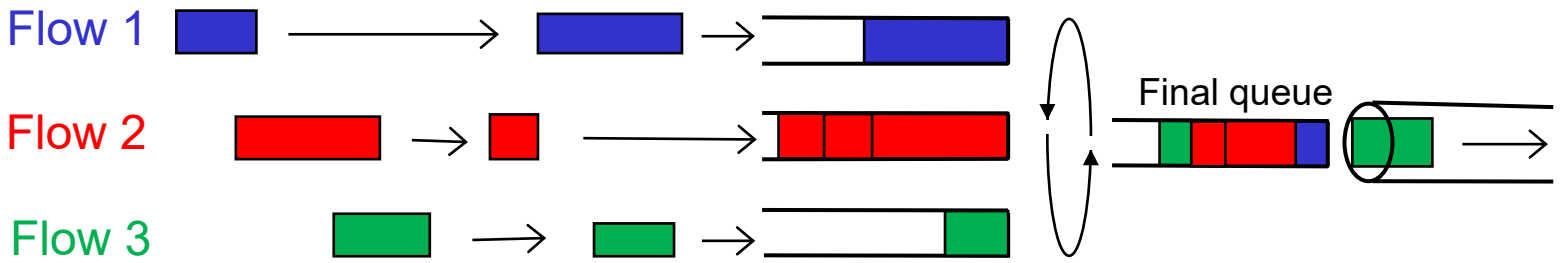
- Algorithms of type round robin are able to implement the weighted max-min fairness principle.
- There is a queue per flow and a final queue where packets are selected at each round of the round robin.
- When a packet is transmitted and the final queue is empty (i.e., the system is in a congested time period), a new round starts:
  - the algorithm moves (in a round robin fashion) some queued packets of each non-empty queue to the final queue,
  - the packets in the final queue are transmitted in the present round.
- The different types of round robin algorithms differ on how the queued packets of each non-empty queue are selected on each round.

Round Robin (RR)



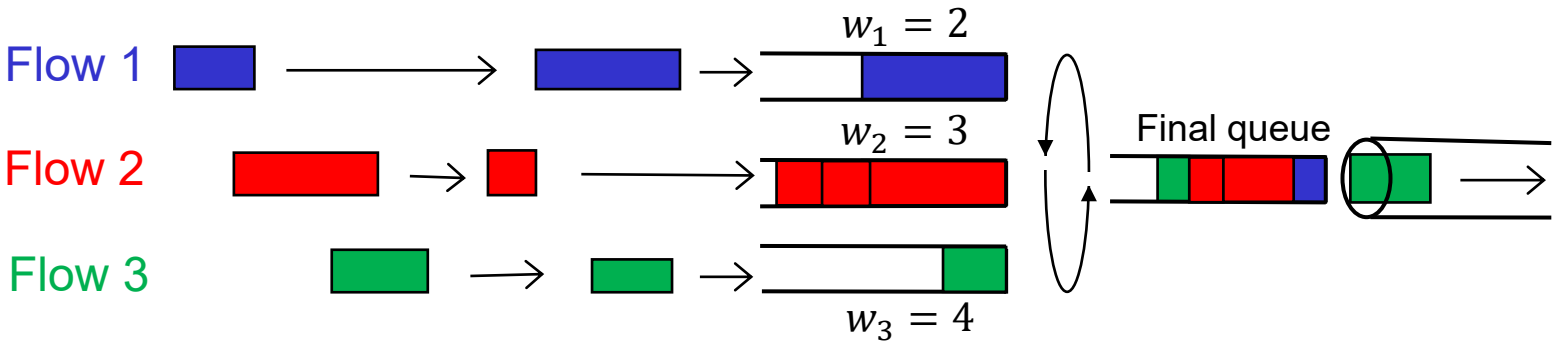
- RR is the simplest of all types of round robin algorithms. At each round:
  - the algorithm moves one single packet from each non-empty queue to the final queue (to be transmitted in the current round).
- It implements the max-min fairness with weight values given by the avg. packet size of each packet flow:
  - in congested time periods, the resources of the link are assigned proportionally to the avg. packet size of the active flows (i.e., the flows of the non-empty queues).
- It does not allow the administrator to configure other weight values per packet flow.

Weighted Round Robin (WRR)



- In WRR, a given number of packets n_i for each flow i is initially configured by the administrator.
- Then, at each round:
  - the algorithm moves up to n_i packets from each non-empty queue to the final queue (to be transmitted by the link in the current round);
  - if a non-empty queue i has less than n_i packets, all queued packets are moved to the final queue.
- It implements the max-min fairness with weight values w_i:
  - the n_i values are computed (next slide) to obtain the desired weights w_i, which must take into account the avg. packet size of each flow.

Weighted Round Robin (WRR)



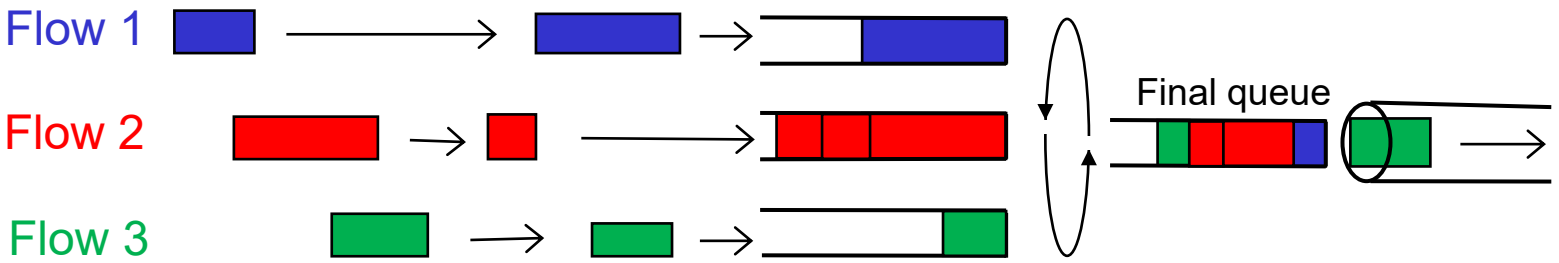
- Consider the following example:
  - the weight values are the ones in the above figure;
  - the avg. packet sizes B_i of flows i = 1, 2 and 3 are 100, 200 and 500 Bytes, respectively.
- First, the normalized weights are computed as $\widetilde{w}_i = w_i/B_i$. Reducing to the same denominator:

$$\widetilde{w}_1 = 2/100 = 20/1000$$
$$\widetilde{w}_2 = 3/200 = 15/1000$$
$$\widetilde{w}_3 = 4/1000$$

- Then, the maximum number of packets n_i of flow i selected per round should be configured as:

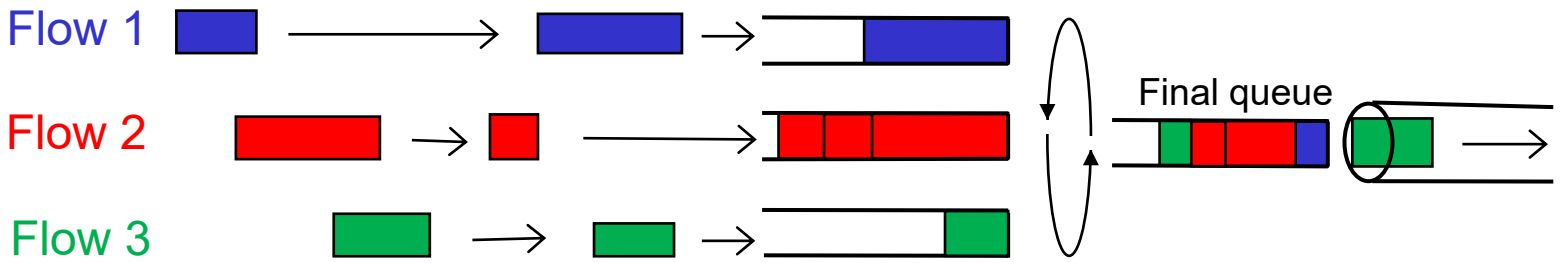
$n_1 = 20$ packets, $n_2 = 15$ packets, $n_3 = 4$ packets

Weighted Round Robin (WRR)



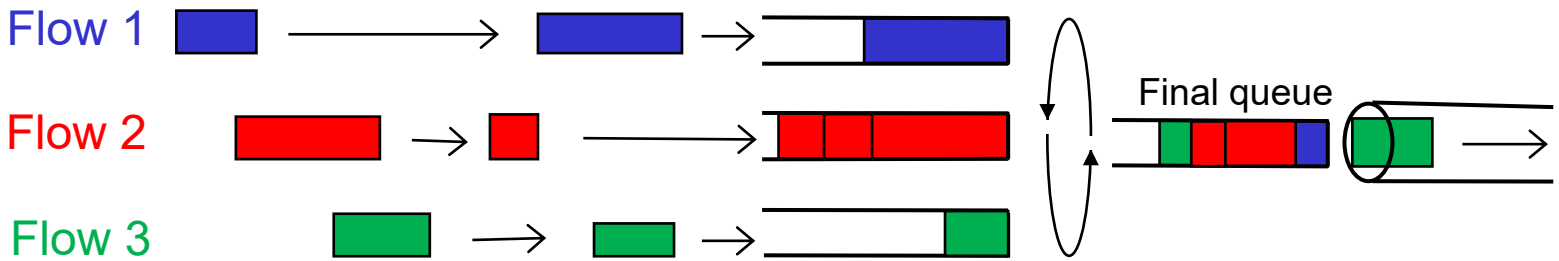
- WRR has three drawbacks:
  - the avg. packet size of each flow must be known in advance
  - when the initial avg. packet size of each flow changes over time, the algorithm's behaviour is no longer is in accordance with the desired weights
  - when n_i values are required to be very large, the link can be transmitting too long packet sequences of each flow, which degrades the *jitter* (*i.e.*, the difference between the maximum and the minimum packet delay) suffered by the packets each flow
- The next round robin type of algorithm (Deficit Round Robin) is more complex but addresses these drawbacks.

Deficit Round Robin (DRR)

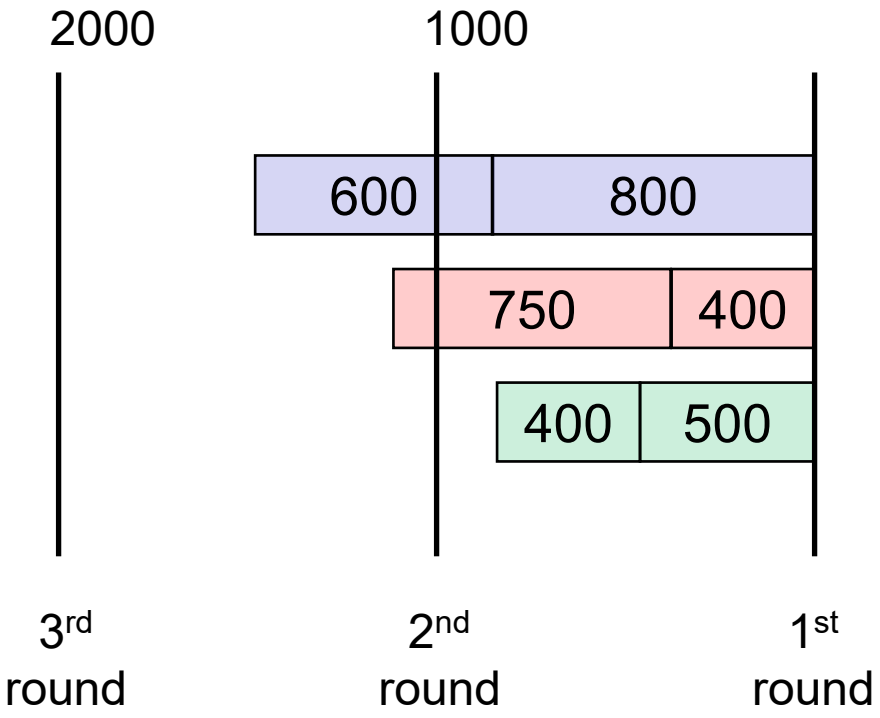


- In DRR, a given quantum value q_i (in Bytes) for each flow i is initially configured. Then, at each round:
  - the algorithm moves (to the final queue) the first packets from each non-empty queue whose total size is not higher than its quantum;
  - the difference between the number of selected bytes and the quantum is computed as a credit for the next round;
  - when a queue is empty, its credit is set to zero.
- It implements the weighted max-min fairness with weights given by the quantum values q_i defined initially for each flow i.
- It is desirable the quantum value q_i of each flow i to be at least its maximum packet size, so that at least one packet from each non-empty flow is transmitted at each round.

Deficit Round Robin (DRR)



$q_1 = q_2 = q_3 = 1000$ Bytes



1st DRR round:

a) 1st packet of queue 1 is transmitted, queue 1 gets a credit of 200

b) 1st packet of queue 2 is transmitted, queue 2 gets a credit of 600

c) both packets of queue 3 are transmitted, queue 3 gets a credit of 100

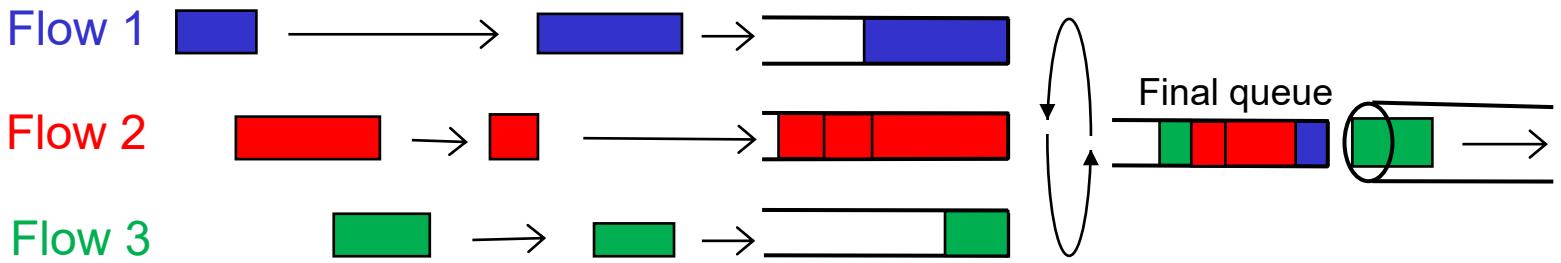
2nd DRR round:

a) 2nd packet of queue 1 is transmitted, queue 1 gets a credit of 600

b) 2nd packet of queue 2 is transmitted, queue 2 gets a credit of 850

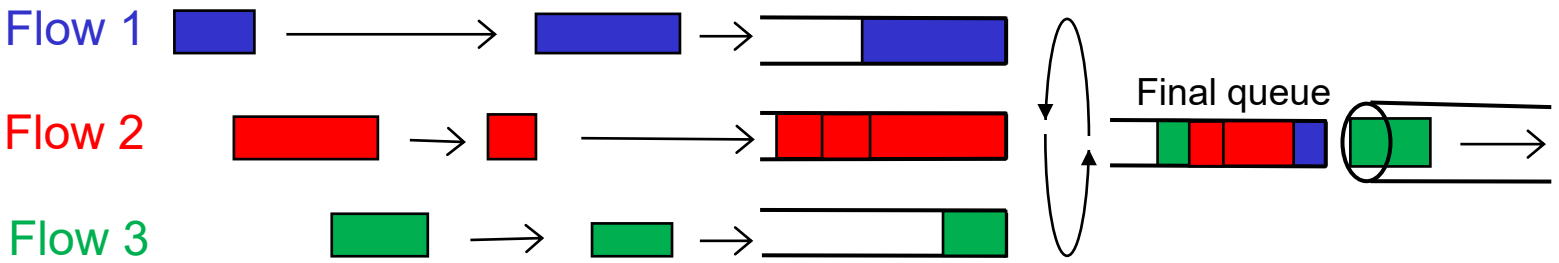
c) queue 3 is empty, its credit becomes 0

Deficit Round Robin (DRR)



- As opposed to WRR, in DRR:
  - the avg. packet sizes of the flows do not need to be known in advance,
  - the algorithm behaviour implements the weighted max-min fairness (with weights given by the quantum values) even when the avg. packet sizes of the flows change with time.
- The considered absolute quantum values are a trade-off between two factors. The lower the quantum values are:
  - the lesser number of packets are selected from each queue at each round, and, therefore, the lesser jitter is introduced on the packets of each flow,
  - but the more computational time is required as there are more frequent DRR rounds to be computed.

Deficit Round Robin (DRR)



- DRR provides weighted max-min fairness between the different flows for longer congested time periods.
- However, it is not perfect for shorter congested time periods.
  - In particular, within each round, packets of queue *i* are transmitted after all packets of queue *i* – 1 and before all packets of queue *i* + 1.
- There are other packet scheduling algorithms able to provide weighted max-min fairness for shorter congested time periods. Examples:
  - Weighted Fair Queuing (WFQ)
  - Self-Clock Fair Queuing (SCFQ)
- Such algorithms are much more complex (requiring more computational power) and only used in very particular cases.